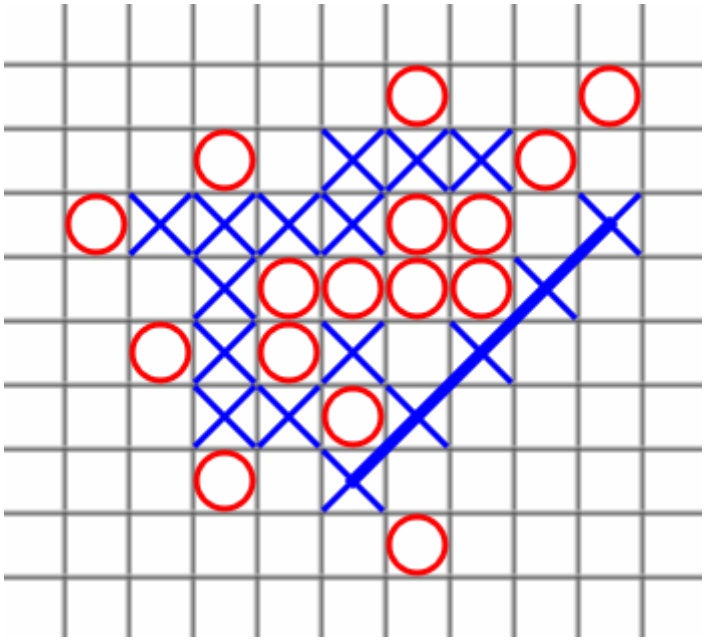
Tradiční piškvorky

Cílem této práce je navrhnout a vytvořit databázovou aplikaci pro hraní známé strategické deskové hry *Piškvorky*. Ze hry bude řešena pouze databázová vrstva aplikace, proto se snažte co nejvíce programových rutin uložit do databáze a také zajistěte jejich automatické spouštění při nastalé události.

Tradiční piškvorky jsou hra pro dva hráče, která se hraje na čtverečkovaném papíře. Ve hře se hráči střídají po jednom tahu, ve kterém každý hráč umístí na volné místo na papíře svoji značku. Obvykle hráči používají symboly kolečko (O) a křížek (X). Vítězí ten hráč, kterému se podaří sestavit nepřerušovanou řadu alespoň pěti svých značek v libovolném směru (vodorovně, svisle, uhlopříčně). Pokud není možné umístit novou značku, hra končí remízou. Hra probíhá na papíře, který bude mít definovanou velikost. Nejmenší rozměr papíru, na kterém bude možné hrát, může mít rozměry 5x5 čtverečků, největší rozměr papíru nesmí překročit velikost 20x20 čtverečků. Také bude možné definovat na kolik značek se bude hrát. Velikost vítězné řady by měla být z intervalu 5 až 15. Je třeba si dát pozor na skutečnost, aby se vítězná řada vešla na definovaný papír.

Také se bude měřit herní čas. Hra začíná umístěním první značky začínajícího hráče. Od tohoto okamžiku se začíná měřit herní čas druhého hráče, a to až do doby, kdy umístí svoji značku. V tento okamžik se začíná měřit herní čas pro začínajícího hráče. Jakmile začínající hráč umístí svoji druhou značku, pozastaví se měření jeho herního času a opětovně se spustí měření času druhého hráče. A tak pořád dokola, dokud hra neskončí výhrou jednoho z hráčů nebo remízou.



Zdroj: `https://cs.wikipedia.org/wiki/Piškvorky`

V relační databázi budou evidována data v těchto tabulkách:

HRA	Každá hra se hraje na novém čistém papíře o dané velikosti na požadovaný počet vítězných symbolů. Všechny tyto hodnoty musí vyhovovat parametrům uvedených v tabulce OMEZENI. Hru hrají dva různí hráči, kde v dané hře jeden z nich umisťuje kolečka a ten druhý křížky. A jeden z nich celou hru zahajuje. Tabulka bude také obsahovat, v jakém stavu se hra nachází a po ukončení hry herní časy obou hráčů. Herní čas bude uveden v sekundách.
HRAC	Každý hráč musí být zaregistrován. Kromě jeho jména budou v tabulce uloženy i jeho herní statistiky, tj. počet vítězství, proher či remíz jak pro hry, ve kterých začínal tak také pro hry, kde hrál jako druhý.
OMEZENI	Tato tabulka slouží jako parametry programu. Těmi jsou šířka čtverečkovaného papíru, výška čtverečkovaného papíru a délka vítězné řady. Každý parametr bude uložen jako jeden záznam tabulky a bude definovat povolený rozsah hodnot pro daný parametr.
STAV	Číselník ukazující, v jakém stavu se může nacházet hra. Stavy mohou být tyto: *rozehraná*, *vítězství začínajícího hráče*, *prohra začínajícího hráče* nebo *remíza*.
TAH	Uchovává informaci, kam umístil svoji značku hráč, který je na řadě v rozehrané hře. Ke každému tahu se bude automaticky ukládat také časová značka, která dočasně nebo trvale zastavuje měření času právě hrajícímu hráči.

Z uložených dat v databázi vytvořte databázové pohledy, které nabídnou tato data:

PAPIR	Zobrazení čtverečkovaných papírů všech dosud odehraných i probíhajících her. Každý řádek papíru bude zobrazen voláním funkce RADEK_PAPIRU. Zobrazení daného čtverečkovaného papíru získáme vhodnou filtrační podmínkou.
REMIZY	Hry, které dospěly do remízy. Obsahuje použité rozměry papíru, velikost vítězné řady, jména hráčů této hry, jméno hráče, který začínal, značky používané hráči a jak dlouho celá hra trvala v sekundách.
PROHRY_ZACINAJICI	Hry, ve kterých prohrál začínající hráč. Obsahuje stejné sloupce, jako pohled REMIZY. Navíc obsahuje informaci, kolik bylo ve hře zahráno tahů.
VYHRY_ZACINAJICI	Hry, ve kterých zvítězil začínající hráč. Obsahuje stejné sloupce, jako pohled REMIZY. Navíc obsahuje informaci, kolik bylo ve hře zahráno tahů.

V databázových pohledech `REMIZY`, `PROHRY_ZACINAJICI` `VYHRY_ZACINAJICI` a `VYHRY_ZACINAJICI` získejte dobu hraní hry jako součet herních dob obou hráčů.

V zápisu kódu databázového pohledu `VYHRY_ZACINAJICI` musí být využita technika *Common Table Expression* (CTE).

V databázi budou uloženy a používány tyto funkce (s parametry):

`SPATNY_PARAMETR` Podle návratové hodnoty funkce poznáme, že je v zakládané hře:

- `0` – vše v pořádku.
- `1` – příliš malý počet řádků na papíru (menší než 5).
- `2` – příliš velký počet řádků na papíru (větší než 20).
- `3` – příliš malý počet sloupců na papíru (menší než 5).
- `4` – příliš velký počet sloupců na papíru (větší než 20).
- `5` – příliš malý počet znaků ve vítězné řadě (menší než 5).
- `6` – příliš velký počet znaků ve vítězné řadě (větší než 15).
- `7` – vítězná řada delší, než šířka papíru.
- `8` – vítězná řada delší, než výška papíru.

`RADEK_PAPIRU` Vrátí řetězec, který odpovídá konkrétnímu řádku papíru dané hry. Pro výpis zvolte v řetězci tyto symboly:

- `X` - značka křížek jednoho hráče dané hry.
- `O` - značka kolečko druhého hráče hry.
- `mezera` - symbol označující volné políčko na papíře.

`HERNI_CAS` Vrátí číslo určující, kolik sekund hrál daný hráč danou hru, tj. sečte rozdíly časových značek, kdy hrál daný hráč a kdy hrál před ním druhý hráč.

`REMIZA` Vrátí hodnotu `TRUE`, pokud daná hra dospěla do remízového stavu, tj. není možné udělat další tah. Jinak vrací hodnotu `FALSE`.

`VYHRA` Vrátí hodnotu `TRUE`, pokud právě hrající hráč v dané hře vyhrál, tj. svým posledním tahem docílil požadované minimální délky vítězné řady svých značek. Jinak vrací hodnotu `FALSE`.

V databázi budou uloženy a používány tyto procedury (s parametry):

`ZABRAN_HRE` Zabrání vytvoření nové hry, pokud je nastaven špatně libovolný parametr hry nebo ve hře bude chtít hrát jeden hráč hrát proti sobě.

`ZABRAN_TAHU` Zabrání tahu, který není možno udělat.

`KONEC_HRY` Spočítá herní časy hráčů právě dokončené hry.

`STATISTIKY` Aktualizuje statistické údaje hráčů, kteří dohráli danou hru.

O automatické činnosti v databázi se postarají triggery o:

- hlídání parametrů nové hry (volání funkce `SPATNY_PARAMETR`),
- hlídání, aby novou hru hráli dva různí hráči,
- hlídání hráčů, aby se ve hře pravidelně střídali po jednom tahu,
- hlídání hráče, aby nepokládal svoji značku na již obsazené místo,
- hlídání hráče, aby nepokládal svoji značku mimo papír,
- hlídání hráče, aby nemohl realizovat tah ve hře, ve které nehraje,
- hlídání hráče, aby nemohl realizovat tah ve hře, která již skončila,
- hlídání a aktualizace stavu hry, tj. zda nedospěla do remízy nebo vítězství jednoho z hráčů (volání funkcí `REMIZA` a `VYHRA`),
- spočítání herní doby hráčů, kteří dohráli aktuální hru (volání funkce `HERNI_CAS`),
- aktualizace statistických údajů hráčů, kteří dohráli aktuální hru.

Konfigurace, spuštění a průběh hry

1. Jednorázová konfigurace databáze:
   a. Naplnění tabulky `OMEZENI` hraničními hodnotami parametrů.
   b. Naplnění tabulky `STAV` daty odpovídající různým stavům hry.
2. Registrace hráčů je provedena vkládáním nových záznamů do tabulky `HRAC`.
3. Hra je zahájena úspěšným vložením nového záznamu do tabulky `HRA`. Neúspěšné vložení např. znamená, že některé parametry hry jsou mimo hraniční hodnoty definované v tabulce `OMEZENI`.
4. Že hra běží, ověříme zobrazením papíru, tj. vypsáním řádků prostřednictvím databázového pohledu `PAPIR` pro aktuálně rozehranou hru.
5. Každý tah ve hře je proveden vložením nového záznamu do tabulky `TAH`. Pokud ke vložení nedošlo, byla aktivována některá z výše uvedených kontrol.
6. Po úspěšném tahu je vhodné si vždy zobrazit aktuální situaci na papíře prostřednictvím databázového pohledu `PAPIR` pro aktuálně rozehranou hru.
7. Pokud hra neskončila, pokračuj bodem 5, jinak bodem 8 (výhra) nebo bodem 9 (remíza). 8. Hra skončila vítězstvím jednoho z hráčů. To je vhodné ověřit voláním pohledu `VYHRY_ZACINAJICI` nebo `PROHRY_ZACINAJICI` a také se podívat na aktualizované statistiky hrajících hráčů v tabulce `HRAC`. Jdi na bod 10.
9. Hra skončila remízou. To je vhodné ověřit voláním pohledu `REMIZY` a také se podívat na aktualizované statistiky hrajících hráčů v tabulce `HRAC`.
10. Novou hru zahájíme bodem 3, s případnou registrací nového hráče bodem 2.

Realizace databázové aplikace

Celá aplikace bude vytvořena v databázovém prostředí Oracle s využitím jazyků SQL a PL/SQL. Pro ukládání databázových objektů dodržujte uvedené názvy, protože formální stránka aplikace bude automaticky ověřována validátorem.

Povinné a volitelné části aplikace

Celá databázová aplikace je rozdělena na povinné a volitelné části. Každá z těchto částí je bodována a dohromady lze získat až 100 bodů. K udělení zápočtu potřebujete získat 51 bodů. Podle toho si můžete naplánovat, jaké části skutečně zrealizujete. Jen pozor na odevzdání po řádném termínu, kdy nabíhá bodová penalizace.

Povinné části aplikace, které budou po formální stránce kontrolovány validátorem:

- fyzický datový model včetně všech integritních omezení. Vše bude napsané v souboru `model.sql`.
- konfigurační data – naplnění tabulek `OMEZENI`, `STAV` a `HRAC`. Vše bude napsané v souboru `konfigurace.sql`.
- uložená funkce `RADEK_PAPIRU`. Kód funkce bude napsán ve skriptu `funkce.sql`.
- databázový pohled `PAPIR`, ve kterém je volána funkce `RADEK_PAPIRU`. Kód pohledu bude napsán ve skriptu `pohledy.sql`.
- databázový pohled `VYHRY_ZACINAJICI`, ve kterém bude použita technika *CTE*. Kód pohledu bude napsán ve skriptu `pohledy.sql`.
- uložená procedura `ZABRAN_TAHU`. Kód procedury bude napsán ve skriptu `procedury.sql`.
- trigger, který bude volat proceduru `ZABRAN_TAHU` v okamžiku, kdy hráč provede neplatný tah. Kód triggeru bude napsán ve skriptu `triggery.sql`.

Povinné části aplikace, které budou kontrolovány vyučujícím:

- PL/SQL skript realizující scénář jedné hry, kterou vyhraje začínající hráč. Hra proběhne bez neplatného tahu. Ve scénáři bude po každém tahu vypsán papír, po skončení hry bude zobrazen výpis pohledu `VYHRY_ZACINAJICI` a výpisy tabulek `HRA` a `HRAC`. Scénář bude uložen v souboru `scenar-vyhra.sql`
- PL/SQL skript realizující scénář, kdy bude korektně založena hra a během hraní občas některý z hráčů vykoná neplatný tah. Zkuste takto otestovat všechny typy neplatných tahů. Celý scénář skončí výhrou začínajícího hráče. Ve scénáři bude po každém tahu vypsán papír, po skončení hry bude zobrazen výpis pohledu `VYHRY_ZACINAJICI` a výpisy tabulek `HRA` a `HRAC`. Scénář bude uložen v souboru `scenar-chybnetahy.sql`.

- ke každému scénáři přiložte log/výpis vykonání jeho příkazů, ze kterého bude patrné, co se během scénáře odehrálo. Tyto logy můžete uložit buď prostředky databázové manageru anebo příkazem `SPOOL` v klientovi `sqlplus`. Soubory se budou jmenovat `scenar-vyhra.log` a `scenar-chybne-tahy.log`.

Volitelné části aplikace, které budou po formální stránce kontrolovány validátorem:

- uložené funkce `SPATNY_PARAMETR`, `HERNI_CAS`, `REMIZA` a `VYHRA`. Vše bude napsané v souboru `funkce.sql`
- databázové pohledy `REMIZY` a `PROHRY_ZACINAJICI`. Vše bude zapsané v souboru `pohledy.sql`.
- uložené procedury `ZABRAN_HRE`, `KONEC_HRY` a `STATISTIKY`. Vše bude zapsané v souboru `procedury.sql`.
- volání funkce `SPATNY_PARAMETR` v proceduře `ZABRAN_HRE`.
- dvě volání funkce `HERNI_CAS` v proceduře `KONEC_HRY`.
- trigger či triggery volající funkce `REMIZA` a `VYHRA`. Vše bude zapsané v souboru `triggery.sql`.
- trigger či triggery volající procedury `ZABRAN_HRE`, `KONEC_HRY` a `STATISTIKY`. Vše bude zapsané v souboru `triggery.sql`.

Volitelné části aplikace, které budou kontrolovány vyučujícím:

- PL/SQL skript realizující scénář jedné hry, kterou prohraje začínající hráč. Hra proběhne bez neplatného tahu. Ve scénáři bude po každém tahu vypsán papír, po skončení hry bude zobrazen výpis pohledu `PRORY_ZACINAJICI` a výpisy tabulek `HRA` a `HRAC`. Scénář bude uložen v souboru `scenar-prohra.sql`.
- PL/SQL skript realizující scénář jedné hry, která skončí remízou. Hra proběhne bez neplatného tahu. Ve scénáři bude po každém tahu vypsán papír, po skončení hry bude zobrazen výpis pohledu `REMIZY` a výpisy tabulek `HRA` a `HRAC`. Scénář bude uložen v souboru `scenar-remiza.sql`.
- PL/SQL skript, který bude zkoušet všechny neplatné pokusy o založení nové hry. Scénář bude uložen v souboru `scenar-chybne-hry.sql`.
- ke každému scénáři přiložte log/výpis vykonání jeho příkazů, ze kterého bude patrné, co se během scénáře odehrálo. Tyto logy můžete uložit buď prostředky databázové manageru anebo příkazem `SPOOL` v klientovi `sqlplus`. Soubory se budou jmenovat `scenar-prohra.log`, `scenar-remiza.log` a `scenar-chybne-hry.log`.

Povinnou součástí semestrální práce bude pochopitelně i dokumentace. Ta bude odevzdána v souboru `dokument.pdf`.

Bonus: Konzolová aplikace
Místo skriptů realizující všech 5 scénářů a jejich logů je možné vytvořit konzolovou aplikaci v jazyce C, která bude využívat techniku vnořeného SQL. Za tuto aplikaci bude možné získat až 25 bodů a uznání vyučujících předmětu.

Odevzdání práce

Výsledný produkt bude odevzdán v podobě archivu pojmenovaného podle šablony *orionlogindba.zip*, kde archiv bude obsahovat tyto soubory. Jiné než uvedené pojmenování souborů a všech dalších objektů bude hodnoceno za nesplněné.

- `model.sql` - ručně napsaný SQL skript, který založí uvedené tabulky včetně povinných integritních omezení.
- `konfigurace.sql` - ručně napsaný SQL skript, který naplní konfiguračními daty tabulky `OMEZENI` a `STAV`. Navíc naplní tabulku `HRAC` alespoň třemi hráči, které budou použity v testovacích scénářích.
- `funkce.sql` - ručně napsaný PL/SQL skript zahrnující kód povinných uložených funkcí a případně kód volitelných uložených funkcí.
- `procedury.sql` - ručně napsaný PL/SQL skript zahrnující kód povinných uložených procedur a případně kód volitelných uložených procedur.
- `pohledy.sql` - ručně napsaný PL/SQL skript zahrnující kód povinných databázových pohledů a případně kód volitelných databázových pohledů.
- `triggery.sql` - ručně napsaný PL/SQL skript zahrnující kód povinných triggerů a případně kód volitelných triggerů.
- `scenar-vyhra.sql` - ručně napsaný PL/SQL skript zahrnující kód scénáře, ve kterém vyhraje začínající hráč.
- `scenar-vyhra.log` - výpis z konzole po spuštění příslušného scénáře.
- `scenar-chybne-tahy.sql` - ručně napsaný PL/SQL skript zahrnující kód scénáře, ve kterém vyhraje začínající hráč, a navíc otestuje všechny možné chybné tahy ve hře.
- `scenar-chybne-tahy.log` - výpis z konzole po spuštění příslušného scénáře.
- `destrukce.sql` - ručně napsaný SQL skript, který zničí všechny databázové objekty aplikace
- `aplikace.sql` - ručně napsaný SQL skript, který vytvoří všechny databázové objekty, naplní konfigurační data a spustí všechny vypracované scénáře včetně jejich logování, nakonec vše zničí.
- `dokument.pdf` - dokumentace uložená ve formátu PDF včetně použitých pramenů.

Navíc může daný archiv obsahovat soubory k volitelným scénářům, kterou jsou popsány výše.

Ohodnocení práce

Každá elementární část aplikace bude náležitě obodována. Každý student musí získat přidělené body z každé povinné části a může získat další body z volitelných částí. Více prozradí následující tabulky.

Povinná část aplikace	**Body**
Datový model včetně všech integritních omezení	5
Konfigurační data tabulky `OMEZENI`	3
Konfigurační data tabulky `STAV`	1
Konfigurační data tabulky `HRAC`	1
Uložená funkce `RADEK_PAPIRU`	5
Databázový pohled `PAPIR`	4
Volání funkce `RADEK_PAPIRU` v pohledu `PAPIR`	1
Databázový pohled `VYHRY_ZACINAJICI`	5
Použití techniky *CTE* v pohledu `VYHRY_ZACINAJICI`	5
Uložená procedura `ZABRAN_TAHU`	5
Trigger volající proceduru `ZABRAN_TAHU`	4
Volání procedury `ZABRAN_TAHU` v triggeru	1
Skript `scenar-vyhra.sql`	4
Log skriptu `scenar-vyhra.log`	1
Skript `scenar-chybne-tahy.sql`	4
Log skriptu `scenar-chybne-tahy.log`	1
Dokumentace ve formátu PDF	1
Celkem	**51**

Pokud bude práce odevzdána po řádném termínu, bude k jejímu hodnocení připočítána penalizace 5 bodů za každý započatý týden zpoždění. Z tohoto důvodu bude nutné naprogramovat některé z volitelných částí pro získání nadpolovičního počtu bodů a splnění nutné podmínky pro zisk zápočtu.

Volitelná část aplikace	**Body**
Uložená funkce `SPATNY_PARAMETR`	1
Uložená funkce `HERNI_CAS`	2
Uložená funkce `REMIZA`	2
Uložená funkce `VYHRA`	5
Databázový pohled `REMIZY`	1

Databázový pohled `PROHRY_ZACINAJICI`	1
Uložená procedura `ZABRAN_HRE`	5
Uložená procedura `KONEC_HRY`	2
Uložená procedura `STATISTIKY`	2
Dvě volání funkce `HERNI_CAS` v proceduře `KONEC_HRY`	2
Volání funkce `SPATNY_PARAMETR` v proceduře `ZABRAN_HRE`	1
Trigger volající funkci `REMIZA`	1
Volání funkce `REMIZA` v triggeru	1
Trigger volající funkci `VYHRA`	1
Volání funkce `VYHRA` v triggeru	1
Trigger volající proceduru `ZABRAN_HRE`	1
Volání procedury `ZABRAN_HRE` v triggeru	1
Trigger volající proceduru `KONEC_HRY`	1
Volání procedury `KONEC_HRY` v triggeru	1
Trigger volající proceduru `STATISTIKY`	1
Volání procedury `STATISTIKY` v triggeru	1
Skript `scenar-prohra.sql`	4
Log skriptu `scenar-prohra.log`	1
Skript `scenar-remiza.sql`	4
Log skriptu `scenar-remiza.log`	1
Skript `scenar-chybne-hry.sql`	4
Log skriptu `scenar-chybne-hry.log`	1
Celkem	**49**

Body uvedené v tabulkách budou přiděleny jen v případě, že odpovídající kód daných částí skutečně dělá to, co dělat má. To by měly potvrdit skripty příslušných scénářů.